PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Architecture and Design Specification

Smart Complaint Management System
Version 1.0
Prepared: 06-10-2026
Team Members: Dhawal Pathak — PES1UG24CS151
GANESH M — PES1UG24CS168
Harshitha P — PES1UG24CS186

1. Introduction

1.1 Purpose

Defines the architecture and detailed design of the Smart Complaint Management System and translates SRS requirements into modules, interfaces, data flow, security controls and testable design elements.

1.2 Scope

Authentication, complaint creation/tracking, administrator management, persistence, authorization and API communication.

2. Architecture

2.1 Goals and Constraints

- Clear separation of presentation, API/application, business and data layers.
- Secure authentication and role-based authorization.
- Simple architecture suitable for a student mini-project.
- Extensible enough to add users/categories without redesign.

2.2 Architecture Pattern and Rationale

Layered architecture is proposed: Presentation → API/Application → Business/Service → Data Access → Database. It provides separation of concerns while avoiding the operational complexity of microservices for this project.

2.3 Component Diagram — Logical Components

Component	Responsibility	Interface
Web Client/UI	User/admin screens	HTTPS API
Authentication Module	Registration/login/session/token	Auth API
Complaint Service	Create/validate/retrieve/update complaints	Complaint API
Admin Service	Privileged listing/filtering/status/resolution	Admin API
Authorization Middleware	Authentication and role checks	Protected routes
Data Access Layer	Database operations	DB driver/ORM
Database	Persistent user/complaint data	DB connection

2.4 Component Descriptions

- Web Client presents role-specific screens.
- Authentication module validates credentials.
- Complaint service owns complaint lifecycle operations.
- Admin service exposes privileged management operations.
- Authorization middleware blocks unauthorized roles.
- Data access isolates database-specific operations.
- Database stores persistent records.

2.5 Traceability to Requirements

SRS Requirements	Architecture Components
FR-01/02	Authentication Module
FR-03/04/05/06	Complaint Service + Data Access
FR-07	Authorization Middleware
FR-08/09/10	Admin Service + Complaint Service
FR-11	Data Access + Database
NFR-03/SEC-01/SEC-03	Authentication/Security
NFR-04/SEC-02	Authorization Middleware

2.6 Security Architecture

Threat/Concern	Control
Spoofing	Authentication and secure session/token validation
Broken access control	Role checks on every admin operation
Injection	Server-side validation and safe parameterized DB operations
Credential disclosure	Password hashing, HTTPS in deployment, externalized secrets
Information exposure	No passwords/secrets in responses or logs
Session/token misuse	Validation and expiration where applicable

2.7 Technology Stack

The supplied project list identifies Python (Flask/Django) or PHP (Laravel) as possible backend choices. The team should select one and keep it consistent. Because the supplied GitHub repository is currently empty, this document does not falsely claim a completed implementation stack.

3. Design

3.1 Design Overview

Requests enter through the web client, pass authentication/authorization where required, are processed by the relevant service and are persisted through the data-access layer.

3.2 Sequence Diagram 1 — Submit Complaint

Step	Component	Interaction
1	User	Open complaint form
2	Web Client	Display fields
3	User	Enter title/category/description and submit
4	Web Client → API	Authenticated POST
5	Auth Middleware	Validate session/token
6	Complaint Service	Validate and create complaint
7	Data Access → DB	Persist complaint with user ID
8	API → Client	Return success/ID
9	Client	Show confirmation/status

3.3 Sequence Diagram 2 — Admin Resolves Complaint

Step	Component	Interaction
1	Admin	Open dashboard
2	Client → Admin API	Request complaint list
3	Auth Middleware	Verify authentication + admin role
4	Admin Service	Retrieve/filter complaints
5	Admin	Choose new status
6	Admin Service	Validate transition/resolution
7	Data Access → DB	Persist update
8	API → Client	Return updated complaint
9	Client	Display updated status

3.4 API Design

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	Register user	No
POST	/api/auth/login	Authenticate	No
POST	/api/complaints	Create complaint	User
GET	/api/complaints	List own complaints	User
GET	/api/complaints/{id}	View details	Owner/Admin
GET	/api/admin/complaints	List/filter all	Admin
PATCH	/api/admin/complaints/	Update status	Admin

Method	Endpoint	Purpose	Auth
	{id}/status		
PATCH	/api/admin/complaints/{id}/ resolution	Record resolution	Admin

3.5 Error Handling, Logging and Monitoring

- 400 invalid input
- 401 unauthenticated
- 403 unauthorized role
- 404 missing/inaccessible resource
- 409 conflicting operation where applicable
- 500 unexpected server failure without stack traces
- Logs should exclude passwords, tokens and secrets.

3.6 UX Design

- Keep complaint submission concise and clearly labelled.
- Show current status prominently.
- Use consistent status labels and success/error feedback.
- Separate user/admin navigation.

3.7 Open Issues and Next Steps

- Finalize backend/database.
- Finalize categories and status workflow.
- Implement modules and APIs.
- Execute Test Plan and update RTM.
- Deploy test instance and measure response time.